Mobile Price Prediction Project

- FEATURE EXTRACTION & ANALYSIS
- YOUR NAME: HARSH RANJAN
- DATE : 29/3/2026

Project Objective

- Predict mobile phone prices using ML models
- Identify key features affecting price
- Help company improve pricing strategy

Dataset Overview

Total records: 541
Features:
RAM, Memory, Battery
Processor, AI Lens
Mobile Height
Price (Target)

[23]:
```
df = pd.read_csv("C:/Users/HP/OneDrive/Desktop/mobile_data.csv")
df.head()
```

[23]:

	Unnamed: 0	Model	Colour	Memory	RAM	Battery_	Rear Camera	Front Camera	AI Lens	Mobile Height	Processor_	Prize
0	0	Infinix SMART 7	Night Black	64	4	6000	13MP	5MP	1	16.76	Unisoc Spreadtrum SC9863A1	7,299
1	1	Infinix SMART 7	Azure Blue	64	4	6000	13MP	5MP	1	16.76	Unisoc Spreadtrum SC9863A1	7,299
2	2	MOTOROLA G32	Mineral Gray	128	8	5000	50MP	16MP	0	16.64	Qualcomm Snapdragon 680	11,999
3	3	POCO C50	Royal Blue	32	2	5000	8MP	5MP	0	16.56	Mediatek Helio A22	5,649
4	4	Infinix HOT 30i	Marigold	128	8	5000	50MP	5MP	1	16.76	G37	8,999

Data Exploration

```
]: df.isnull().sum()
```

```
]: Unnamed: 0       0
   Model            0
   Colour           0
   Memory           0
   RAM              0
   Battery_         0
   Rear Camera      0
   Front Camera     0
   AI Lens          0
   Mobile Height    0
   Processor_       0
   Prize            0
   dtype: int64
```

```
df = df.drop_duplicates()
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541 entries, 0 to 540
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Unnamed: 0      541 non-null    int64
 1   Model           541 non-null    object
 2   Colour          541 non-null    object
 3   Memory          541 non-null    int64
 4   RAM             541 non-null    int64
 5   Battery_        541 non-null    int64
 6   Rear Camera     541 non-null    object
 7   Front Camera    541 non-null    object
 8   AI Lens         541 non-null    int64
 9   Mobile Height   541 non-null    float64
 10  Processor_      541 non-null    object
 11  Prize           541 non-null    object
dtypes: float64(1), int64(5), object(6)
memory usage: 50.8+ KB
```

Data Preprocessing

```
[10]: df = pd.get_dummies(df, drop_first=True)
```

```
[28]: df = df.drop("Unnamed: 0", axis=1)
```

```
[29]: df.columns
```

```
[29]: Index(['Model', 'Colour', 'Memory', 'RAM', 'Battery_', 'Rear Camera',
       'Front Camera', 'AI Lens', 'Mobile Height', 'Processor_', 'Prize'],
      dtype='object')
```

```
[33]: df.columns = df.columns.str.strip()
```

```
[34]: print(df.columns)
```

```
Index(['Model', 'Colour', 'Memory', 'RAM', 'Battery_', 'Rear Camera',
       'Front Camera', 'AI Lens', 'Mobile Height', 'Processor_', 'Prize'],
      dtype='object')
```

```
[36]: y = df["Prize"]
      X = df.drop("Prize", axis=1)
```

```
[38]: X = pd.get_dummies(X, drop_first=True)
```

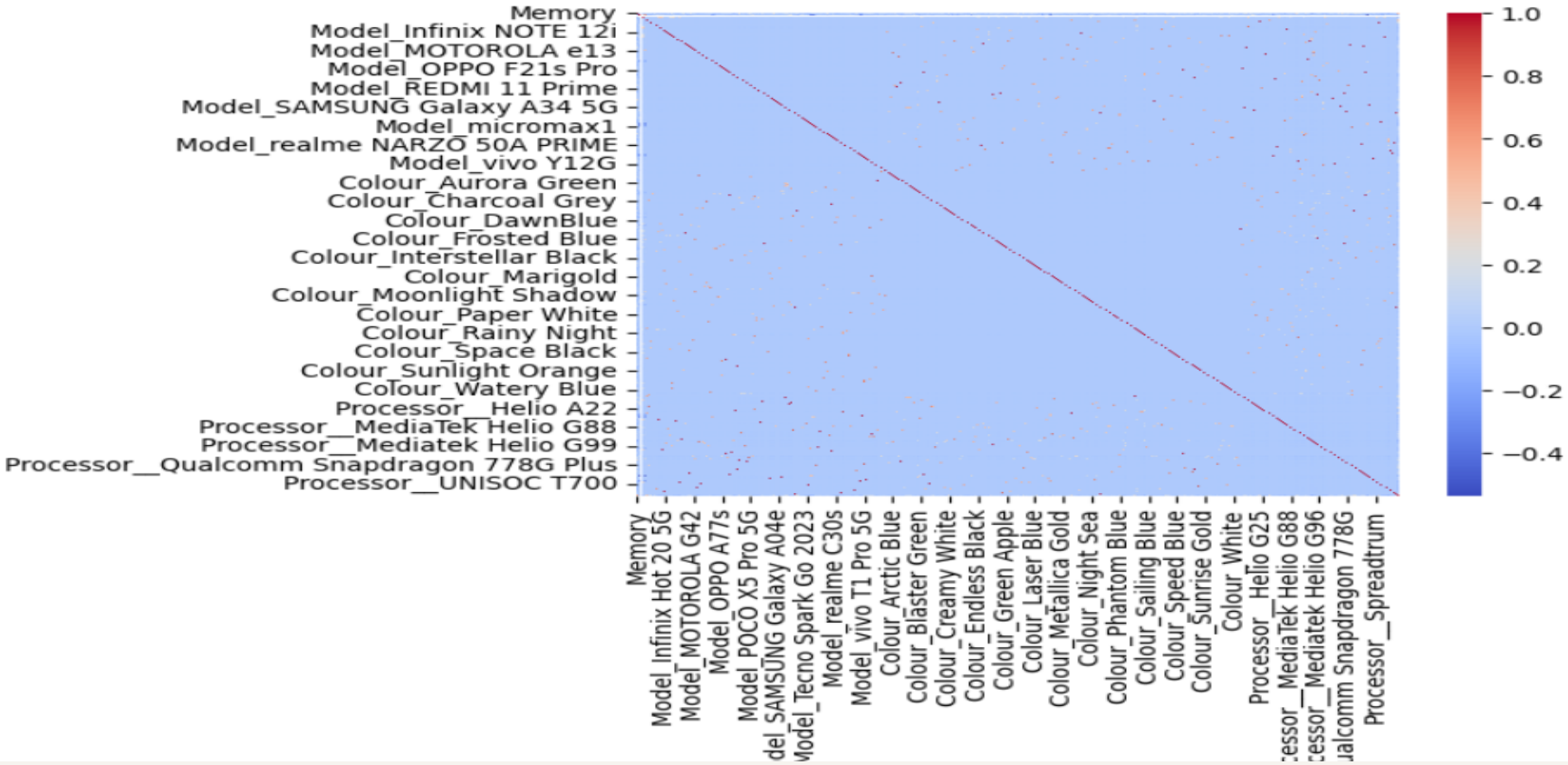
```
[39]: df_encoded = pd.concat([X, y], axis=1)
```

```
[43]: print(df.columns)
```

```
Index(['Model', 'Colour', 'Memory', 'RAM', 'Battery_', 'Rear_Camera',
       'Front_Camera', 'AI_Lens', 'Mobile_Height', 'Processor_', 'Prize'],
      dtype='object')
```

Heatmap

```
y = df["Prize"]
X = pd.get_dummies(df.drop("Prize", axis=1), drop_first=True)

df_final = pd.concat([X, y], axis=1)

sns.heatmap(df_final.corr(), cmap="coolwarm")
plt.show()
```



Train Test Split

[53]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

[54]:
```
from sklearn.ensemble import RandomForestRegressor

model = RandomForestRegressor()
model.fit(X_train, y_train)
```

[54]:
```
▾ RandomForestRegressor
RandomForestRegressor()
```

[55]:
```
y_pred = model.predict(X_test)
```

[56]:
```
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print("MAE:", mae)
print("RMSE:", rmse)
```

```
MAE: 2053.6644986893843
RMSE: 5707.993890119216
```

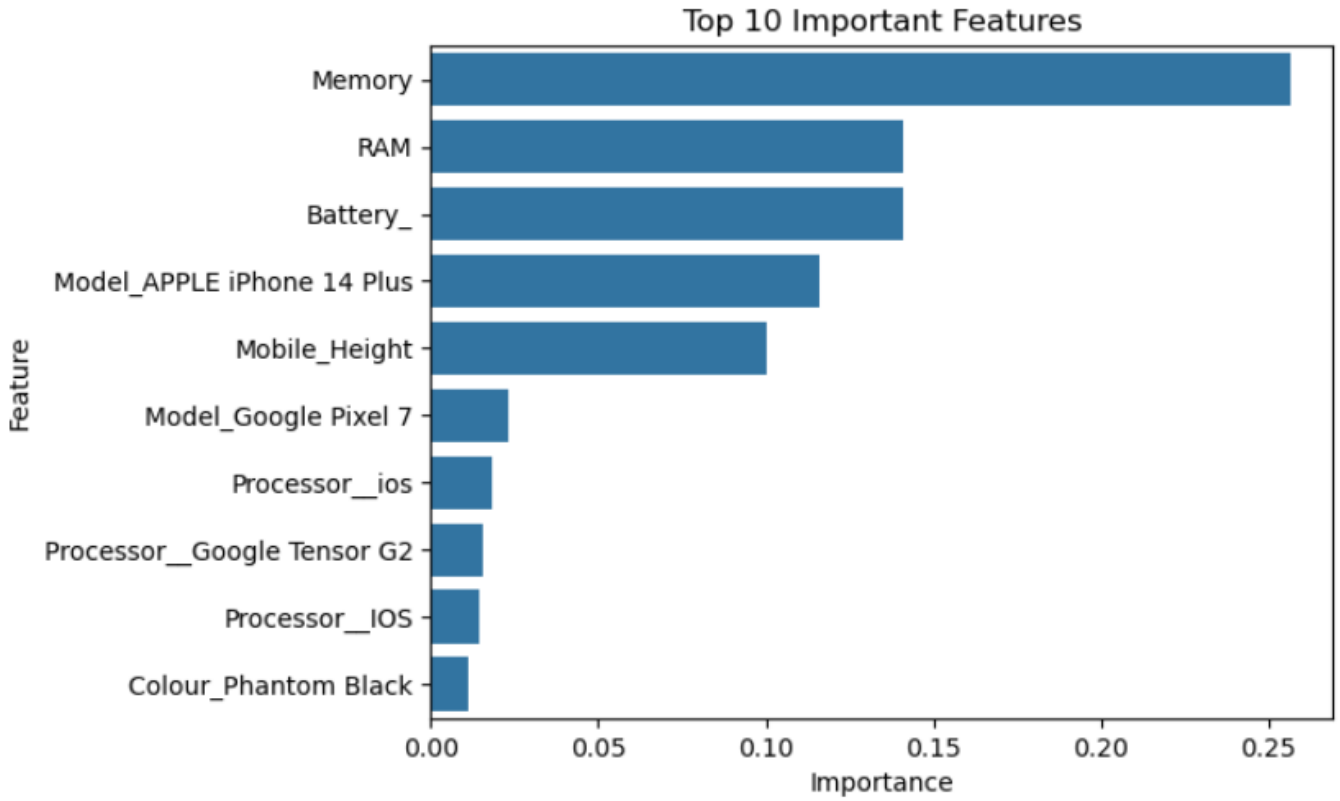
FEATURE IMPORTANCE

[57]:
```
import pandas as pd

feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
}).sort_values(by="Importance", ascending=False)

print(feature_importance.head(10))
```

```
                          Feature  Importance
0                          Memory    0.256567
1                             RAM    0.141192
2                        Battery_    0.141148
8     Model_APPLE iPhone 14 Plus    0.116248
6                   Mobile_Height    0.100528
10          Model_Google Pixel 7    0.022909
587               Processor__ios    0.018014
482  Processor__Google Tensor G2    0.015367
489               Processor__IOS    0.014362
373         Colour_Phantom Black    0.010982
```

TOP 10 IMPORTANT FEATURES

[58]:
```
import seaborn as sns
import matplotlib.pyplot as plt

sns.barplot(x="Importance", y="Feature", data=feature_importance.head(10))
plt.title("Top 10 Important Features")
plt.show()
```



COMPARING MULTIPLE MODELS

```python
from sklearn.model_selection import cross_val_score
import numpy as np

for name, model in models.items():
    scores = cross_val_score(
        model, X, y, cv=5,
        scoring='neg_mean_absolute_error'
    )

    print(f"{name} CV MAE:", abs(scores.mean()))
```

```
Linear Regression CV MAE: 4667.888991079891
Decision Tree CV MAE: 3256.4596330275235
Random Forest CV MAE: 3156.6994989806317
```

```python
best_model = min(
    [(name, abs(cross_val_score(model, X, y, cv=5,
     scoring='neg_mean_absolute_error').mean()))
     for name, model in models.items()],
    key=lambda x: x[1]
)

print("Best Model:", best_model)
```

```
Best Model: ('Decision Tree', np.float64(3085.468433571186))
```

How much each feature contributes to predicting Price

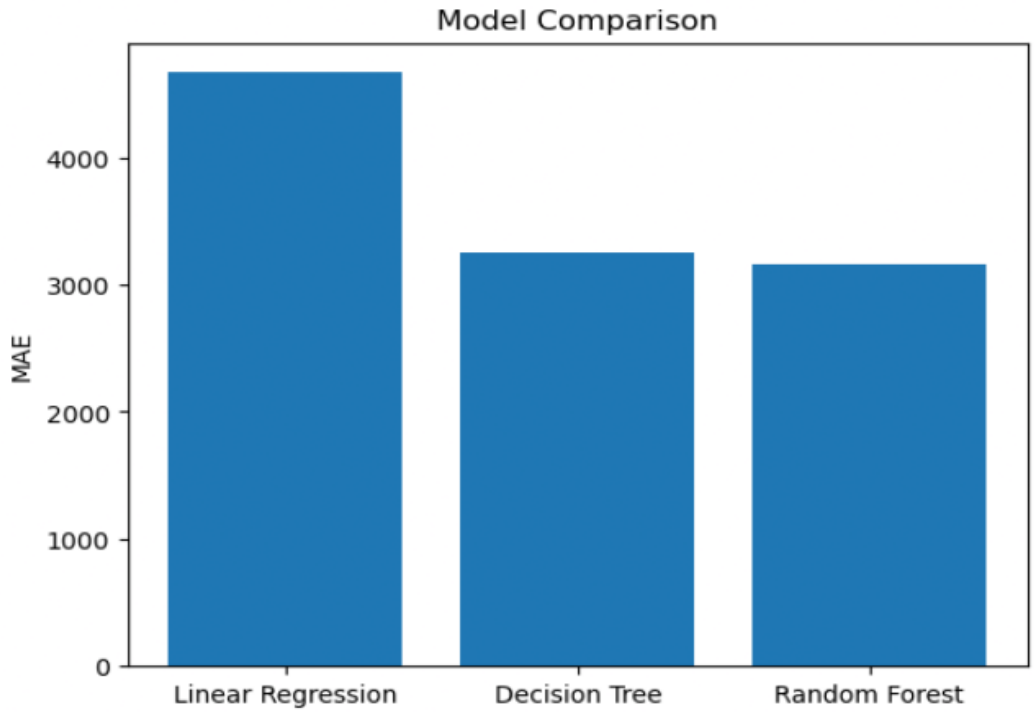
[84]:
```
rf = RandomForestRegressor()
rf.fit(X_train, y_train)

import pandas as pd

importance = pd.Series(rf.feature_importances_, index=X_train.columns)
importance.sort_values(ascending=False).head(10)
```

[84]:
```
Memory                          0.243047
RAM                             0.149671
Battery_                        0.139662
Mobile_Height                   0.092246
Model_APPLE iPhone 14 Plus      0.088976
Model_Google Pixel 7            0.032653
Processor__Google Tensor G2     0.024543
Processor__ios                  0.019722
Processor__6 Core               0.017789
Colour_Midnight                 0.015474
dtype: float64
```

Top 3 Features Responsible in Driving Price -:
1.Memory
2.RAM
3.Battery

Model Performance Comparison

```
]: import matplotlib.pyplot as plt

   names = ["Linear Regression", "Decision Tree", "Random Forest"]
   scores = [4667, 3256, 3156]

   plt.bar(names, scores)
   plt.ylabel("MAE")
   plt.title("Model Comparison")
   plt.show()
```



This slide compares the performance of different models using MAE to identify the most accurate model.

RECOMMENDATIONS

- Increase focus on RAM & Memory in premium phones
- Performance Score strongly impacts pricing
- Processor type also influences price

CONCLUSION

This project demonstrates end-to-end machine learning pipeline including data cleaning, feature engineering, model evaluation, and business recommendations.

Random Forest is the most effective model for this dataset .

THANK YOU

This is a sample text. Insert your desired text here. This is a sample text
Insert your desired text here. Insert your desired text here